

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for **Movie Night**, a group movie-selection application. It is intended to guide the design and implementation of the system and to serve as a reference for scope, expected behavior, and constraints throughout development. This document does not describe internal system architecture, database schemas, or API contracts — those are covered in the accompanying System Design document.

1.2 Scope

Movie Night is a web/mobile application that allows users to discover movies, maintain a personal watchlist, and collaboratively select a movie to watch with a group in real time. Users can register and log in, search for movies via integration with The Movie Database (TMDB) API, and manage a personal watchlist by adding, removing, and rating movies. Users can create or join a real-time voting session using a 6-digit session code; within a session, participants vote synchronously on movies drawn from the group's combined watchlists until either a unanimous choice is reached or the session ends and the most-voted movie is selected. The application also provides an AI Assistant that recommends a movie to a user based on a short conversational exchange about their mood and preferences.

The following features are explicitly excluded from the scope of this version of the system:

- Push notifications and email notifications.
- Payment processing, subscriptions, or any monetization features.
- Movie streaming or playback functionality. The application facilitates collaborative movie selection only.
- Social networking features beyond collaborative session participation, including friend lists, direct messaging, comments, and public user profiles.
- Asynchronous or independent movie voting. All voting activities occur synchronously within an active session.
- Offline functionality.

1.3 Definitions

Term	Definition
Session	A temporary, real-time group activity in which participants vote together on movies to reach a shared decision. Identified by a 6-digit code and destroyed once it ends.
Leader	The user who creates a session. Only the Leader can start the session. The session continues even if the Leader disconnects after it has started.

Member	Any user who joins a session using its code (including the Leader).
Watchlist	A per-user list of movies they intend to watch, stored via the Movie Service.
Pool	The set of candidate movies generated by combining all session participants' watchlists. Duplicate movies are removed before the collection is randomized for the voting process.
Vote	A member's yes/no response to the currently displayed movie during a session.
Session Code	A randomly generated 6-digit code used to join a session. Becomes invalid once the session ends.
Selected Movie	The movie a session concludes with — either by unanimous agreement or by the fallback rule (highest vote count, tie broken randomly).
AI Assistant	An LLM-backed feature that asks a user a short series of questions and returns a single movie suggestion based on their answers.

2. Overall Description

2.1 Product perspective

Movie Night is a new, self-contained system built on a microservices architecture, not an extension of an existing product. Clients communicate with the backend exclusively through an API Gateway, which handles authentication and routes requests to four independent services: User Service, Movie Service, Session Service, and AI Assistant Service. Each service owns its own data — Session Service uses Redis for ephemeral session state, Movie Service integrates with the TMDB API for movie data, and the AI Assistant Service integrates with an LLM API for conversational recommendations. This document describes system *behavior* only; service boundaries, schemas, and API contracts are detailed in the System Design document.

2.2 User classes

The system has a single user class: **authenticated user**. There is no separate account type for session leaders or members — "leader" and "member" are transient *roles* a user holds within a specific session, not distinct user types with different permissions elsewhere in the system. Any authenticated user can create a session (becoming its leader) or join one (becoming a member), and the same user may hold different roles across different sessions. This distinction matters for the data model: role does not belong on the User entity, it belongs on the session-participant relationship.

2.3 Assumptions and constraints

- The system assumes availability of the TMDB API for movie search, details, and posters; movie-related features are degraded or unavailable if TMDB is unreachable.

- The system assumes availability of an external LLM API for the AI Assistant; the assistant is unavailable if this dependency is down.
- No offline mode is supported — the application requires an active internet connection for all core features.
- The system is deployed to a single region for this version; no multi-region redundancy or failover is assumed.
- Real-time session functionality assumes a persistent WebSocket connection between client and server; users with unstable connections may experience delayed session updates or be treated as non-responsive within a voting round.
- Session state is ephemeral and stored in Redis; if Redis data is lost mid-session, the session cannot be recovered and must be restarted.
- The system is designed for group sizes typical of casual movie-night use (a small number of participants per session); it is not designed or tested for large-scale concurrent sessions.

3. Functional Requirements

3.1 Authentication

- The system shall allow a new user to register with an username and password.
- The system shall reject registration if the username is already in use.
- The system shall store passwords using a secure one-way hash, plaintext passwords shall never be stored.
- The system shall allow a registered user to log in with username and password.
- The system shall issue a JWT upon successful login, to be used for authenticating subsequent requests.
- The system shall reject requests to protected endpoints that lack a valid, non-expired JWT.
- The system shall allow an authenticated user to log out, invalidating their session on the client side.

3.2 User profile

- The system shall allow an authenticated user to view their own profile (watchlist and watched history).

3.3 Movie search & discovery

- The system shall allow an authenticated user to search for movies by title via the TMDB API.
- The system shall display movie details (title, poster, overview, release year, rating) for a selected search result.

3.4 Watchlist management

- The system shall allow an authenticated user to add and remove a movie to their personal watchlist.

- The system shall allow an authenticated user to view their own watchlist.

3.5 Watched history

- The system shall allow an authenticated user to mark a movie as watched and give it a rating.
- The system shall allow an authenticated user to view their own watched history.

3.6 Session creation & joining

- The system shall allow an authenticated user to create a session, becoming its Leader.
- The system shall generate a unique, random 6-digit code for each newly created session.
- The system shall allow an authenticated user to join an existing session by entering a valid session code.
- The system shall allow only the Leader to start a session.
- The system shall keep a session active if the Leader disconnects after the session has started.

3.7 Synchronized voting flow

- The system shall generate the movie pool for a session as the deduplicated combining of all participants' watchlists, shuffled into random order, upon session start.
- The system shall broadcast the current movie in the pool to all session participants in real time via WebSocket.
- The system shall allow each participant to cast a yes/no vote on the currently displayed movie.
- The system shall advance to the next movie in the pool once all participants have voted on the current movie, or once a voting timeout has elapsed.

3.8 Session termination & result

- The system shall end the session and declare the current movie as the Selected Movie if all participants vote yes on it.
- The system shall end the session once the pool is exhausted without unanimous agreement, selecting the movie with the highest vote count as the Selected Movie.
- The system shall broadcast the Selected Movie (including its poster) to all participants when the session ends.
- The system shall invalidate the session code once the session ends.

3.9 AI Assistant recommendation

- The system shall present an authenticated user with a short series of preference-related questions (e.g. mood, desired feeling).
- The system shall send the user's answers to an LLM API to generate a movie suggestion.

- The system shall display the suggested movie's details (title, poster, overview) to the user.
- The system shall handle LLM API failures gracefully, informing the user that a suggestion is temporarily unavailable rather than leaving the interaction unresolved.

4. Non-Functional Requirements

4.1 Performance

- WebSocket broadcast of session events (current movie, vote updates, session results) shall reach all connected participants within 1 second under normal network conditions.
- TMDB API responses (search, details, posters) shall be cached with a TTL of 24 hours to reduce redundant external calls and latency on repeated lookups.

4.2 Scalability

- Session state shall be stored in Redis rather than in-memory within a single service instance, so that any Session Service instance can serve any active session (required for horizontal scaling, even if only one instance is deployed for the MVP).

4.3 Security

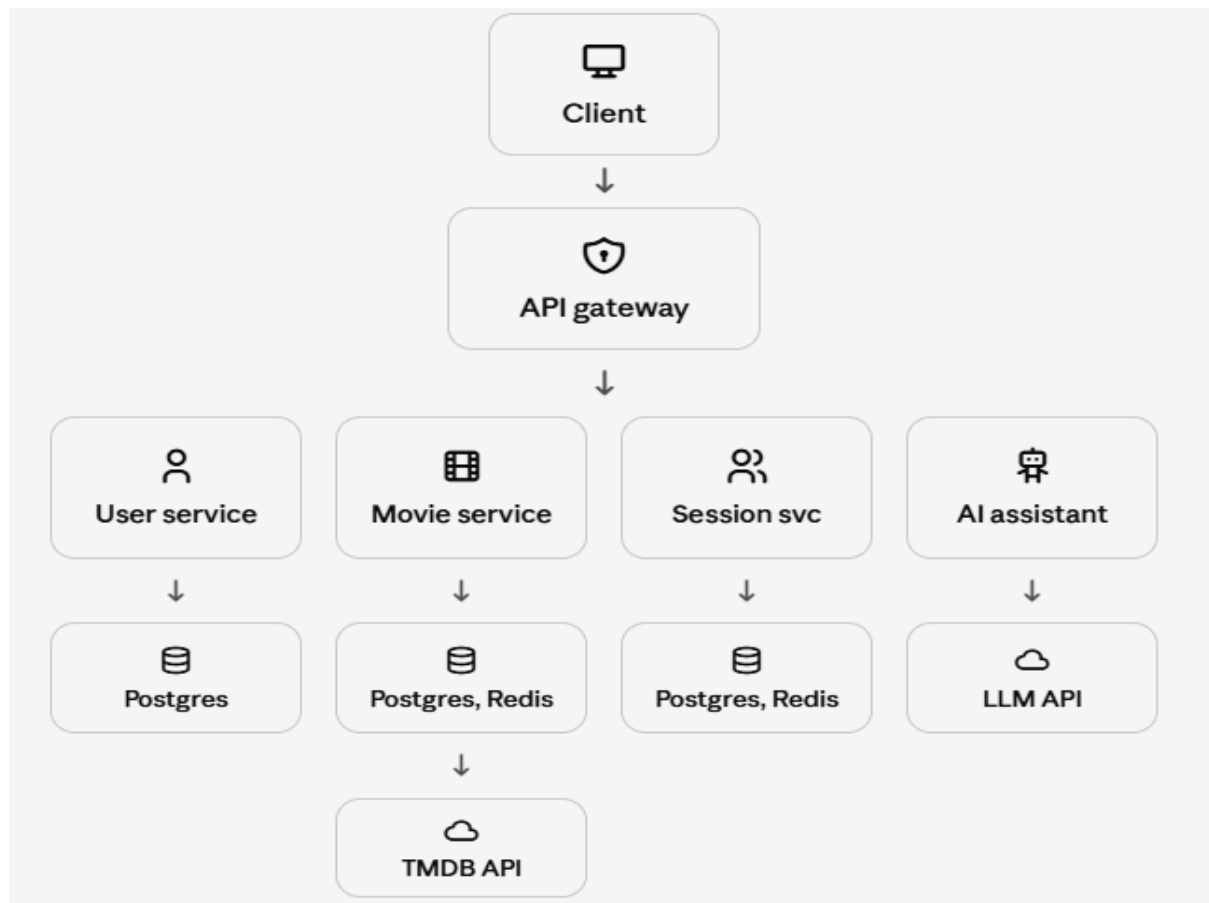
- JWTs shall expire after a defined period (e.g. 24 hours) and require re-authentication after expiry.
- Passwords shall be hashed using a secure, industry-standard algorithm before storage; plaintext passwords shall never be stored or logged.
- Session codes shall be rate-limited per IP/user to prevent brute-force guessing of the 6-digit code.
- All API endpoints (except registration and login) shall require a valid JWT.
- Communication between client and server shall use HTTPS/WSS (encrypted transport) in production.

4.4 Availability & reliability

- If the TMDB API is unreachable, previously cached movie data shall remain accessible; only new/uncached lookups shall be degraded.
- If the LLM API is unreachable, the AI Assistant feature shall fail gracefully with a user-facing message rather than hanging or crashing the request.
- If a participant's WebSocket connection drops mid-session, the system shall allow them to reconnect and resume participation in the same session without restarting it.

5. System Design

5.1 High-level architecture



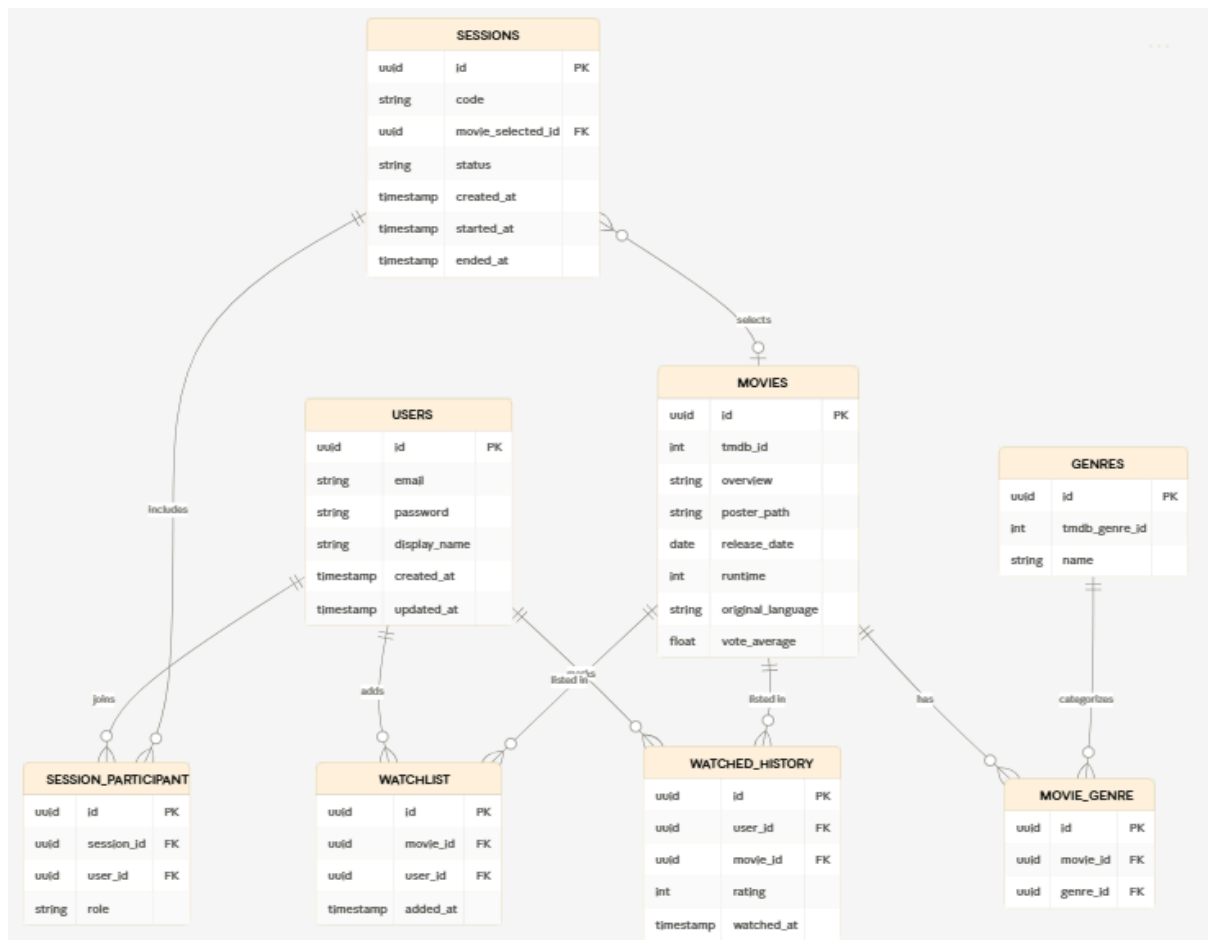
Movie Night follows a microservices architecture behind a single entry point. Clients — web or mobile — never talk to individual services directly; every request goes through the API Gateway, which is responsible for validating the JWT and routing the request to the correct downstream service.

Four independent services sit behind the gateway, each owning a distinct area of responsibility:

- User Service — registration, login, JWT issuance, profile
- Movie Service — TMDB search/details, watchlist, watched history
- Session Service — session creation/joining, real-time synchronized voting, termination logic
- AI Assistant Service — conversational Q&A flow with the LLM API

Each service is independently deployable and owns its own data — there is no shared database between services. Cross-service communication (if ever needed, e.g. Session Service needing watchlist data from Movie Service) happens through internal API calls, not direct database access. This detail — along with each service's actual data store, external dependency, and API contract — is covered in Section 2 (Data Models) and Section 3 (API Contracts) rather than this diagram, which is intentionally kept at the routing level only.

5.2 Entity-Relationship Diagram (ERD)



The relational data model spans eight entities across PostgreSQL, reflecting the boundaries described in Section 5.1. Each service owns only the tables relevant to its responsibility — there is no cross-service foreign key access; all relationships shown exist within a single service's database.

Users is the central identity entity, referenced by Watchlist, WatchedHistory, and SessionParticipants — but never holds a role or session-specific attribute itself, consistent with the decision that "leader" and "member" are roles scoped to a session, not user account types (see Section 2.2).

Movies, Genres, and MovieGenres form a local mirror of TMDB movie data (a many-to-many relationship between movies and genres). This local copy exists so that watchlist and watched-history entries remain resolvable even if TMDB or the Redis cache is temporarily unavailable — TMDB and Redis remain the primary source for *fresh* data, but the local Movies table guarantees previously-seen movies are never permanently lost from the system.

Watchlist and WatchedHistory are both many-to-one relationships from Users and Movies, capturing a user's personal list of movies to watch and movies already watched, respectively. WatchedHistory.rating allows a user to optionally rate a movie once marked as watched.

Sessions holds the permanent record of a voting session — its 6-digit code, lifecycle status, and (once concluded) a nullable foreign key to the `Movies` table identifying the final selected movie. This table is written to at session creation and updated at completion; it does not store live voting state, which remains in Redis for the duration of an active session (per Section 5.1) and is discarded once the session ends.

SessionParticipants is a join table between `Sessions` and `Users`, additionally storing each participant's `role` (leader or member) — since `role` is scoped to a specific session rather than being a permanent user attribute.